

1 Polynomial Hierarchy

Recall the verifier characterizations of NP and coNP. Roughly speaking,

$$\begin{aligned} L \in \text{NP} : x \in L &\iff \exists w \in \{0, 1\}^{p(|x|)} M(x, w) = 1, \\ L \in \text{coNP} : x \in L &\iff \forall w \in \{0, 1\}^{p(|x|)} M(x, w) = 1, \end{aligned}$$

for some polynomial-time Turing machine M and polynomial p . The main difference is the quantifier over the polynomial-length witness: NP begins with $\exists$, while coNP begins with $\forall$.

Example 1: Independent Set.

$$\text{INDSET} = \{(G, k) : G \text{ has an independent set of size at least } k\},$$

we can write $(G, k) \in \text{INDSET}$ as “there **exists** an independent set S of size k .” Thus the problem has an existential witness and is in NP: $\exists S M(G, k, S) = 1$.

Example 2: Exact Independent Set.

$$\text{EXACT-INDSET} = \{(G, k) : \text{the largest independent set of } G \text{ has size exactly } k\}.$$

To certify that the largest independent set has size exactly k , we need to say that there **exists** an independent set S of size k , while **every** set S' of size $k+1$ is not independent. Thus the statement has the form $\exists S \forall S' M(G, k, S, S') = 1$, for a polynomial-time machine M . This motivates allowing several alternating quantifiers over polynomial-length strings.

Definition: Polynomial Hierarchy. [AB09] For $i \geq 1$, a language L is in Σ_i^p if there exists a polynomial-time Turing machine M and a polynomial q such that

$$x \in L \iff \exists u_1 \in \{0, 1\}^{q(|x|)} \forall u_2 \in \{0, 1\}^{q(|x|)} \cdots Q_i u_i \in \{0, 1\}^{q(|x|)} M(x, u_1, \dots, u_i) = 1,$$

where the quantifiers alternate, and $Q_i = \exists$ when i is odd and $Q_i = \forall$ when i is even.

Similarly, $L \in \Pi_i^p$ if the alternating quantifiers begin with $\forall$ instead of $\exists$. Thus a Π_i^p statement has the form $\forall u_1 \exists u_2 \forall u_3 \cdots Q_i u_i$.

The polynomial hierarchy is $\text{PH} = \bigcup_{i \geq 1} \Sigma_i^p$.

Remark. At the first level, we recover the familiar classes $\Sigma_1^p = \text{NP}$ and $\Pi_1^p = \text{coNP}$. The problem EXACT-INDSET above is in Σ_2^p , since its natural description has the form $\exists S \forall S'$. Another example from the textbook is MIN-DNF, where $\phi \in \text{MIN-DNF}$ iff for every smaller DNF formula ψ , there exists an assignment s such that $\phi(s) \neq \psi(s)$. Hence $\text{MIN-DNF} \in \Pi_2^p$.

The classes form a hierarchy $\Sigma_1^p \subseteq \Sigma_2^p \subseteq \Sigma_3^p \subseteq \dots \subseteq \text{PH}$, and similarly for the Π_i^p classes. Intuitively, each new level permits one additional alternation of quantifiers. It is conjectured that every inclusion is strict, i.e., $\Sigma_i^p \subsetneq \Sigma_{i+1}^p$ for every i .

If there is some i such that $\text{PH} = \Sigma_i^p$, then we say that the polynomial hierarchy *collapses to the i th level*. In this case, additional quantifier alternations above level i do not give additional computational power.

Theorem 1 (Collapse of the Polynomial Hierarchy).

1. For every $i \geq 1$, if $\Sigma_i^p = \Pi_i^p$, then $\text{PH} = \Sigma_i^p$. That is, the hierarchy collapses to the i th level.
2. If $\text{P} = \text{NP}$, then $\text{PH} = \text{P}$; that is, the hierarchy collapses to P .

Proof sketch. The key idea in both statements is to group all but the outermost quantifier into a lower-level predicate, and then simplify its leading quantifier.

For the first statement, suppose $\Sigma_i^p = \Pi_i^p$ and consider $L \in \Sigma_{i+1}^p$. Membership in L has the form $\exists u_1 [\forall u_2 \exists u_3 \dots Q_{i+1} u_{i+1} M(x, u_1, \dots, u_{i+1}) = 1]$. For fixed (x, u_1) , the expression inside the brackets is a Π_i^p predicate. By assumption, it can instead be expressed as a Σ_i^p predicate, whose leading quantifier is $\exists$. This leading existential quantifier can be merged with the outer $\exists u_1$, so $L \in \Sigma_i^p$. Hence the $(i+1)$ st level adds no power, and repeating the same argument collapses all higher levels to Σ_i^p .

For the second statement, if $\text{P} = \text{NP}$, then since P is closed under complement, $\text{P} = \text{NP} = \text{coNP}$, so $\Sigma_1^p = \Pi_1^p = \text{P}$. Applying the first statement with $i = 1$, all higher levels collapse to P recursively. Thus $\text{PH} = \text{P}$. $\square$

2 Williams' Theorem

Theorem 2 (Williams' Theorem). [Wil25] For every time-constructible $T(n) \geq n$,

$$\text{DTIME}(T(n)) \subseteq \text{SPACE}\left(\sqrt{T(n) \log T(n)}\right).$$

In class, we prove a simpler but weaker bound of $\text{SPACE}\left(\sqrt{T(n)}(\log T(n))^{3/2}\right)$. The proof reduces a time-bounded TM computation to an instance of the Tree Evaluation Problem.

Tree Evaluation. A Tree Evaluation instance is a rooted tree of height h and fan-in at most d . Each leaf contains a b -bit value, and each internal node v is associated with a function $f_v : \{0, 1\}^{db} \rightarrow \{0, 1\}^b$. The value of an internal node is obtained by applying f_v to the values of its children. The goal is to compute the value of the root.

The straightforward recursive algorithm uses $O(dbh)$ space, since values from different levels of recursion must be stored simultaneously. Cook and Mertz give a much more space-efficient algorithm [CM24]. We use the following weaker form, which is sufficient for the proof:

$$\text{TreeEval}(d, b, h) \in \text{SPACE}((db + h) \log(db)).$$

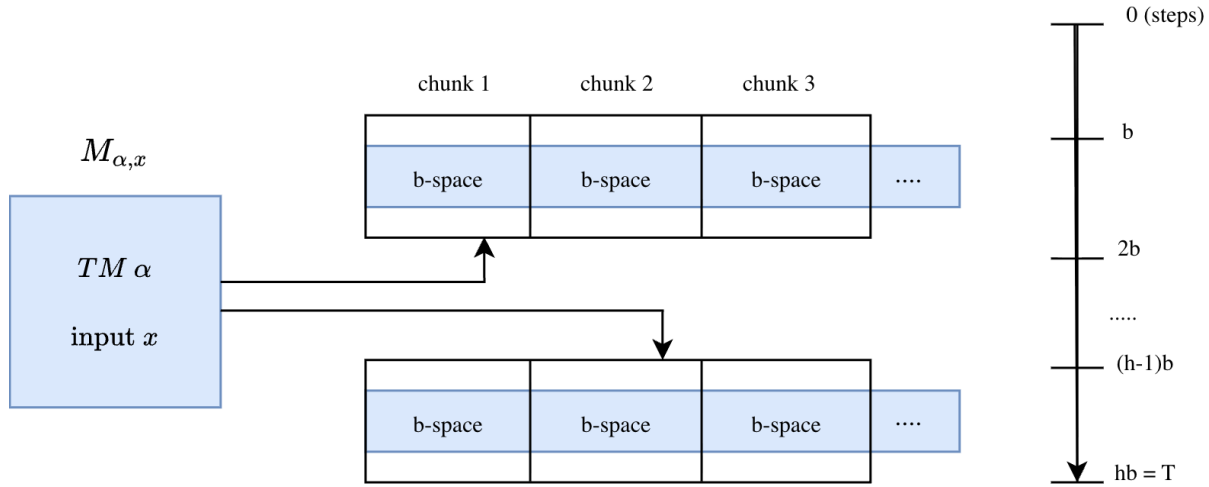


Figure 1: Space and Time Blocking in the Oblivious Two-Tape Simulation

Oblivious simulation. We first transform the computation using the oblivious simulation of Pippenger and Fischer [PF79].

Theorem 3 (Oblivious Turing Machine). *A Turing machine running in time T can be simulated by an oblivious Turing machine U in time $\hat{T} = O(T \log T)$, where the location of each tape head at time t depends only on t and the input length, not on the contents of the input.*

Let $M_\alpha(x)$ be a two-tape Turing machine running in time T .¹ let $U(\alpha, x)$ be its oblivious two-tape simulation. Thus U runs in time $\hat{T} = O(T \log T)$.

Choose a block length b . Divide each tape into blocks of b consecutive cells, and divide the $\hat{T}$ -step computation into $h = \lceil \hat{T}/b \rceil$ time blocks, each containing at most b steps.

During one time block, each tape head moves by at most b positions, so it can access at most two adjacent tape blocks. Since there are two tapes, at most four tape blocks are accessed during any time block.

Computation graph. We create one node for each time block. There is a directed edge (i, j) , with $i < j$, if information produced in time block i is needed to simulate time block j . This occurs either when $i = j - 1$, which provides the machine state and head positions from the previous time block, or when a tape block used in time block j was most recently accessed in time block i .

Each node therefore has in-degree at most 5: one edge from the previous time block and at most four edges corresponding to the four tape blocks that may be active during the current block.

For a time block i , let $\text{config}(i)$ contain the state of the machine, the two head positions, and the contents of all tape blocks accessed during that time block. There are at most four such tape blocks, so $\text{config}(i)$ can be represented using $b' = O(b + \log T) = O(b)$ bits, assuming $b \geq \log T$.

¹Why not just 1-tape TM? 1-tape TM is universal computational-wise, but the time complexity can be quadratic. This is because read and write needs to traverse through the tape (moving back and forth to read and write takes quadratic time. see Sudan's Lecture 8 note section 2 for an example. <https://people.seas.harvard.edu/~madhusudan/MIT/ST07/scribe/lect08.pdf>

Given the contents of all predecessor nodes, we can compute $\text{congif}(i)$ by simulating the machine for the next b steps. Thus each node behaves as a function from at most $5b'$ input bits to b' output bits.

The computation graph is a directed acyclic graph. We unfold it into a tree by duplicating a computation-graph node whenever it is reached along multiple paths. The root computes the content of the final time block, and the leaves correspond to the initial configuration. The resulting Tree Evaluation instance has fan-in $d = 5$, bit-length $b' = O(b)$, and height $h' = O(h)$.

The role of obliviousness is important: since the tape-head positions depend only on time and input length, the edges of the computation graph can be determined without storing or executing the entire computation.

Putting the parameters together. The parameters of the reduction are

$$\hat{T} = O(T \log T), \quad b = \Theta(\sqrt{\hat{T}}), \quad h = \Theta(\hat{T}/b),$$

with $d = 5$, $b' = O(b)$, and $h' = O(h)$. Hence $b, h = O(\sqrt{T \log T})$.

Applying the Tree Evaluation algorithm gives

$$\begin{aligned} O((db' + h') \log(db')) &= O(\sqrt{T \log T} \cdot \log T) \\ &= O(\sqrt{T} (\log T)^{3/2}). \end{aligned}$$

This proves the weaker version stated above.

Acknowledgement

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [CM24] James Cook and Ian Mertz. Tree evaluation is in space $o(\log n \cdot \log \log n)$. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1268–1278, 2024.
- [PF79] Nicholas Pippenger and Michael J. Fischer. Relations among complexity measures. *Journal of the ACM*, 26(2):361–381, 1979.
- [Wil25] R. Ryan Williams. Simulating time with square-root space. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, pages 13–23, 2025.